DJANGO PROJECT #2 · DESIGN SPECIFICATION

Khoj

A disaster missing-persons registry and remains-matching system — interface design, working model, and a ten-phase build roadmap.

Modelled on the August 2026 Bhotekoshi / Rasuwa flood, Nepal
Stack Django 5.1 · SQLite → PostgreSQL · Bootstrap 5
Location D:\Djangoprojects\project#2
Prepared 31 August 2026 · for Abhinav

1 · The problem this models

On 26 August 2026 a glacier collapsed on the north face of Langtang-Lirung. The landslide registered as a magnitude 5.2 event and sent a debris flow down the Bhotekoshi and Trishuli valleys, devastating Rasuwa and Nuwakot districts and villages across the Tibetan side of the border. Within five days the toll had passed 800 dead with roughly 3,000 people still missing.

The engineering problem is not the flood. It is what happened afterwards. Reporting from the following week describes families travelling hundreds of kilometres between hospitals, morgues, police stations and army camps across six districts — Rasuwa, Nuwakot, Chitwan, Nawalpur, Tanahun and Pokhara — because **no single registry of the missing or of the recovered existed**. Unidentified remains were displayed on screens in hospital corridors, annotated with clothing, height and physical features. Around a thousand people a day filed past photographs of 102 bodies in Pokhara alone. One man carried a photograph printed off Facebook and showed it to police, over and over.

A searcher, quoted in the Kathmandu Post:

> **THE REQUIREMENT, STATED PLAINLY**
>
> “How many places are we supposed to visit to search for one missing person? It would be easier for families like ours if bodies were kept in one place.”

Families without money, mobility or an internet connection could not search at all. They waited at home. That is the inequity the system exists to remove: the search should cost a form, not a bus fare across six districts.

What Khoj does

Two populations of records are created independently, by people who will never meet, and the system brings them together:

Family files missing-person report → Matching engine scores every pair ← Responder files unidentified record

Ranked candidates → Human verifier confirms or rejects → **Family notified by a person**

The machine never decides. It only reduces three thousand comparisons to a handful worth a trained person's attention. Everything after that is human.

> **GROUND RULE FOR THIS BUILD**
>
> This is a **reference design and portfolio project running on seeded, fictional data**. It is not deployed as a public service and does not accept real reports. A student-built site that looks like an official registry during a live disaster splinters the search into one more place to check, collects the personal data of people at their most vulnerable, and becomes a magnet for donation fraud impersonating it. The disclaimer stays in the footer of every page. If a relief organisation ever wants this, that is a conversation with them — not a solo launch.

2 · Who uses it, and what they may see

Access control is the spine of this project, not a feature bolted on at the end. Every screen in section 4 exists inside exactly one of these roles. The `role` field already sits on your `accounts.User` model.

ROLE	WHO THEY ARE	WHAT THE SYSTEM IS FOR THEM
Family self-registered	A relative or friend searching for someone. Possibly on a borrowed phone, on a bad connection, under extreme stress.	A form, and then honest status. They file once and stop travelling.
Responder account issued by admin	Hospital, morgue, police post or army camp staff who have custody of unidentified remains.	A fast intake form. They are busy and the data quality of the whole system depends on them.
Verifier account issued by admin	A trained official who decides whether a candidate pair is the same person, and who breaks the news.	A ranked queue and a side-by-side comparison screen. This is the only role that sees both sides at once.
Administrator	Runs the deployment. Issues responder and verifier accounts.	User management, the audit log, and aggregate statistics.

Permission matrix

Build this table into tests. It is the specification that the code must satisfy.

CAPABILITY	PUBLIC	FAMILY	RESPONDER	VERIFIER	ADMIN
View landing page & aggregate counts	●	●	●	●	●
File a missing-person report	—	●	●	—	●
Read a missing-person report	—	**own**	—	●	●
Edit / withdraw a report	—	**own**	—	—	●
File an unidentified record	—	—	●	—	●
Read an unidentified record	—	—	**own org**	●	●
View post-mortem photographs	—	—	**own org**	●	—
See the candidate-match queue	—	—	—	●	●
Confirm or reject a match	—	—	—	●	—
Read the audit log	—	—	—	—	●
Create responder / verifier accounts	—	—	—	—	●

Three rules the matrix encodes

Families never browse the dead. There is no public gallery of unidentified remains and no family-facing search across them. A family sees a photograph of a body only when a trained verifier has already judged it a probable match and has chosen to show them, in a controlled screen. Public browsing is how a system built to help ends up traumatising thousands of people at scale.

Responders are scoped to their own organisation. A nurse in Pokhara has no business reading Chitwan's records. This is a queryset filter, and it is the single most common place a Django project leaks data — the object exists, the URL is guessable, and nobody checked ownership in the detail view.

Only a verifier changes a match's state. The engine may propose. It may never conclude. Every transition is written to the audit log with the actor, the timestamp and the reason.

3 · The data model

Six models. The interesting one is `MatchCandidate`, which is not just a join table — it carries its own data, so it must be a real model rather than a bare `ManyToManyField`.

MODEL	PURPOSE AND THE FIELDS THAT MATTER
User accounts	*Built.* `AbstractUser` plus `role`, `phone`, `organisation`.
Organisation	Hospital, morgue, police post, army camp. `name`, `kind`, `district`, coordinates. Responders belong to one; it is what scopes their queryset.
MissingPersonReport	The family's side. Identity (`full_name`, `also_known_as`, `age`, `sex`, `height_cm`, `build`), last-known position (`last_seen_at` datetime, `last_seen_location`, district, coordinates), appearance (`clothing_description`, `distinguishing_marks`), `reporter` FK to User, `status`, and photographs.
UnidentifiedRecord	The responder's side. Deliberately parallel: `estimated_age_min` / `estimated_age_max`, `sex`, `height_cm`, `build`, `recovered_at` datetime, `recovery_location` + district + coordinates, `clothing_description`, `distinguishing_marks`, `organisation` FK, `custody_reference`, `status`.
MatchCandidate the through model	One row per *pair the engine thought worth looking at*. FK to each side, plus `score`, `score_breakdown` (JSON — which rule contributed what), `state`, `reviewed_by`, `reviewed_at`, `reviewer_notes`. Unique together on the pair.
AuditEvent	Append-only. `actor`, `action`, `object_type`, `object_id`, `ip_address`, `timestamp`, `detail` JSON. Written on every read of a photograph and every state change. Never updated, never deleted.

WHY MATCHCANDIDATE CANNOT BE A MANYTOMANYFIELD

A plain `ManyToManyField` stores only “these two are linked”. Here the link itself has properties — how confident, who reviewed it, when, and why. The moment a relationship needs its own fields, it becomes a model in its own right, wired in with `through=`. This is the single most useful ORM concept in the project and you will meet it in Phase 4.

The comparable fields

The two record types were designed together so that matching is possible at all. Every row below is a field pair the engine can reason about. The asymmetry is the whole difficulty: a family knows a name and an exact age; a mortuary knows a range and a measurement.

MISSING-PERSON REPORT	UNIDENTIFIED RECORD	COMPARISON
`age` — exact, stated	`estimated_age_min/max`	Does the stated age fall inside the estimated band?
`sex`	`sex`	Equality, but a near-gate rather than a hard one — estimates are wrong.
`height_cm` — often a guess	`height_cm` — measured	Absolute difference, tolerant band.
`clothing_description`	`clothing_description`	Token overlap. High value: clothing is specific and memorable.
`distinguishing_marks`	`distinguishing_marks`	Token overlap. Highest value: scars, tattoos and old fractures are near-unique.
`last_seen_at` + location	`recovered_at` + location	Is the recovery point plausibly downstream, within a plausible interval?

4 · How the matching actually works

A scoring function, not a machine-learning model. Every point is traceable to a rule a human can read out loud in a courtroom, which matters enormously here. The output is a number from 0 to 100 and a breakdown explaining it.

SIGNAL	MAX POINTS	RULE
Distinguishing marks	**25**	Token overlap on scars, tattoos, dental work, amputations, old fractures. Near-unique when present, so weighted highest.
Clothing	**20**	Token overlap after normalising colour and garment words. “red kurta” against “maroon kurta” should score partially, not zero.
Age	**18**	Full points inside the estimated band; decaying outside it. A 34-year-old against a 30–40 estimate is a match; against 60–70 it is not.
Geography & time	**15**	Recovery point downstream of last-seen point, within a plausible drift interval. Upstream recovery scores zero — water only goes one way.
Height	**12**	Full points within ±4 cm, decaying to zero at ±15 cm. Family estimates are unreliable, hence the modest weight.
Sex & build	**10**	Equality on sex, partial on build. Soft, because post-mortem estimates are frequently revised.

THRESHOLDS

- **Below 30** — discarded, no row written. Otherwise three thousand reports against a thousand records is three million rows.
- **30 to 55** — stored as a weak candidate, visible in the queue's low-confidence tab.
- **Above 55** — surfaced at the top of the verifier's queue.
- **Any distinguishing-marks hit above 15 points** — always surfaced regardless of total. A tattoo match outranks every other disagreement.

> **WHERE THE REAL DIFFICULTY LIVES**
>
> Nepali names transliterate into Latin script a dozen ways — Shrestha / Shreshtha / Sreshtha, Bishnu / Vishnu. Exact string comparison fails constantly. This is why the project moves to PostgreSQL in Phase 8, for trigram similarity and full-text search. Note that names only help when an unidentified record acquires a partial name later — the initial match is made on body and circumstance, never on name.

Match lifecycle

SUGGESTED → UNDER_REVIEW → FAMILY_CONTACTED → CONFIRMED

↳ any state may fall to REJECTED or SUPERSEDED

STATE	SET BY	MEANING
`SUGGESTED`	the engine	Scored above threshold. Nobody has looked yet.
`UNDER_REVIEW`	verifier	Claimed, so two verifiers do not duplicate work or contact the same family twice.
`FAMILY_CONTACTED`	verifier	Reached out. Awaiting identification by the family or by a physical process.
`CONFIRMED`	verifier	Same person. Closes both records and supersedes every other candidate on both sides.
`REJECTED`	verifier	Not the same person, with a mandatory reason. The pair is never re-surfaced.
`SUPERSEDED`	system	Auto-closed because the other side was confirmed elsewhere.

Confirmation must run inside a database transaction: closing both records, superseding siblings on both sides, writing the audit event and queuing the notification either all happen or none do. A half-confirmed match is a family told twice, or never.

5 · The interface, screen by screen

Wireframes, not visual design — structure and hierarchy first. Bootstrap 5 handles the skin. What matters below is what appears on each screen and, more importantly, what deliberately does not.

5.1 · Public landing page

KHOJ — Sign in · Register

Search for a missing person in one place

File one report. Hospitals, morgues and police posts across every district are searched for you.

File a missing-person report | I am a responder

REPORTS FILED	RECORDS HELD	IDENTIFICATIONS MADE	FACILITIES CONNECTED
2,847	1,102	318	44

How it works — three steps, illustrated. No records shown.

Note what is absent: no search box over the dead, no gallery, no names. Counts only. The public surface of this system is a form and an explanation — nothing that can be scraped, screenshotted, or turned into a rumour.

5.2 · Filing a report — a four-step wizard

KHOJ · New report — Sunita G. · Family

1 · Who | 2 · Where last seen | 3 · Appearance | 4 · Contact

FULL NAME: Ram Bahadur Tamang

ALSO KNOWN AS: nicknames, spelling variants

AGE: 34

SEX: Male ▾

HEIGHT (CM): ~168

BUILD: Medium ▾

Not sure of a value? Leave it blank. A blank field is far better than a guess — a wrong number pushes the right match down the list.

Save and continue | Save draft

Why four steps and not one long form: the person filling this in is distressed and may be on a phone. Short screens with saved drafts survive interruption; a forty-field page does not. Django gives you this with a form-per-step and the session, or a formtools wizard.

KHOJ · New report — Step 3 of 4

1 · Who	2 · Where	**3 · Appearance**	4 · Contact

WHAT WERE THEY WEARING WHEN LAST SEEN?

red and black checked shirt, blue jeans, brown leather sandals

SCARS, TATTOOS, DENTAL WORK, OLD INJURIES, ANYTHING UNUSUAL

tattoo of a bird on left forearm; missing upper right front tooth

This section matters most. Details like these identify people when nothing else can. Write everything you remember, even if it feels small.

Photograph — drag or tap clear, face visible	Add another	Add another

The interface teaches the user what the engine needs. Distinguishing marks carry 25 points and the copy says so in plain language. Design that hides its own scoring produces worse data.

5.3 · Family dashboard and report status

KHOJ · My reports — Sunita G. · Family

Ram Bahadur Tamang, 34 — UNDER REVIEW

Filed 27 Aug · Last seen Timure bazaar, Rasuwa · Ref KHJ-2847

27 Aug 14:20 — Report received and entered into the registry
27 Aug 14:21 — Automatic search begun across 44 connected facilities
29 Aug 09:05 — **A possible match is being reviewed by an official.** You will be contacted directly by phone. Please do not travel.

Contact: Verification desk, Rasuwa · 01-XXXXXXX

Kamala Tamang, 61 — SEARCHING

Filed 27 Aug · No candidates yet · Ref KHJ-2848

The hardest screen in the project. It must be honest without being cruel and specific without being false. A family is told *that* a match is under review, never *which* record, and never shown a photograph here. The line “please do not travel” is the entire point of the system, written into the interface. Note also “Searching” rather than “No match found” — the same fact, without the finality.

5.4 · Responder intake

KHOJ · New unidentified record | Dr K. Adhikari · Pokhara Academy of Health Sciences

CUSTODY REFERENCE: PAHS-UN-104
RECOVERED AT: 28 Aug 2026, 07:10
RECOVERY LOCATION: Trishuli, below Betrawati bridge

EST. AGE RANGE: 30 — 40
SEX: Male ▾
HEIGHT (CM): 169
BUILD: Medium ▾

CLOTHING RECOVERED WITH: checked shirt (red/black), denim trousers
DISTINGUISHING FEATURES: tattoo, bird, left forearm; upper right incisor absent

Photograph — restricted

Uploaded images are stored outside the public media root and served only to authorised roles through a permission-checked view. Every access is logged.

Save record | Save and add another

Optimised for speed under load. Same field vocabulary as the family form so the two are genuinely comparable, “save and add another” because these arrive in batches, and a custody reference so the physical body can always be found again from the digital record.

5.5 · Verifier queue

KHOJ · Match queue | B. Rai · Verifier

New 34 | Mine 6 | Low confidence 91 | Closed 318

KHJ-2847 ↔ PAHS-UN-104 — 86
Ram Bahadur Tamang, 34 · Rasuwa → est. 30–40 M · Betrawati

MARKS 25/25 · CLOTHING 18/20 · AGE 18/18 · GEO 13/15 · HEIGHT 8/12 · SEX 4/10

Open comparison | Claim

KHJ-2311 ↔ CHT-UN-047 — 41
Sita Gurung, 22 · Nuwakot → est. 20–30 F · Chitwan

The score breakdown is shown, never just the number. A verifier who can see that 25 of the 86 points came from a tattoo will trust it; one shown only “86” is being asked to trust a black box with a life-altering decision. This is also what makes the system auditable after the fact.

5.6 · Side-by-side comparison — the decision screen

KHOJ · Comparison · KHJ-2847 ↔ PAHS-UN-104 — Score 86 · Under review by you

REPORTED MISSING

Ram Bahadur Tamang

34 · Male · ~168 cm · medium

Family-supplied photograph

Clothing red and black checked shirt, blue jeans, brown leather sandals
Marks tattoo of a bird on left forearm; missing upper right front tooth
Last seen 26 Aug ~08:00, Timure bazaar, Rasuwa

UNIDENTIFIED RECORD

PAHS-UN-104

est. 30–40 · Male · 169 cm · medium

Restricted photograph
access logged

Clothing checked shirt (red/black), denim trousers
Marks tattoo, bird, left forearm; upper right incisor absent
Recovered 28 Aug 07:10, Trishuli below Betrawati — 31 km downstream

Agreements — bird tattoo, left forearm · missing upper right tooth · red/black checked shirt · age inside band · 1 cm height difference · recovery point downstream within 43 h
Disagreements — sandals not recovered with the body

Confirm identification | Reject — reason required | Request more information | Release claim

Agreements and disagreements are computed and listed separately. Humans anchor on confirming evidence and skim past what contradicts — surfacing the disagreements explicitly is a deliberate guard against that. “Reject” demands a written reason, because that text is what stops the same wrong pair being reconsidered and what an inquiry will read a year later.

5.7 · Administrator

KHOJ · Audit log — admin

actor ▾ | action ▾ | object ▾ | date range

```
29 Aug 09:05:12   b.rai   MATCH_CLAIMED   MatchCandidate#812   10.4.2.19
29 Aug 09:05:44   b.rai   PHOTO_VIEWED   UnidentifiedRecord#104   10.4.2.19
29 Aug 09:11:03   b.rai   MATCH_CONFIRMED   MatchCandidate#812   10.4.2.19
29 Aug 09:11:03   system   MATCH_SUPERSEDED ×4   MatchCandidate#[813,877,901,944]
29 Aug 09:11:04   system   NOTIFICATION_QUEUED   MissingPersonReport#2847
```

Read-only, append-only, and filterable. Every photograph view is a row. In a system holding images of the dead, knowing who looked is not paranoia — it is the control that makes holding them defensible at all.

6 · Build roadmap

Ten phases. Each one ends with something that runs, and each is chosen so that a specific piece of Django is unavoidable rather than optional. Build them in order — later phases genuinely depend on earlier ones.

Phase 0 · Foundation — COMPLETE

Project package `khoj`, apps `accounts` and `registry`, custom user model with roles, `.env` secrets, base template, Bootstrap.

CONCEPTS `AbstractUser` and inheritance · `AUTH_USER_MODEL` and why it must precede the first migrate · `TextChoices` · two-level URL routing · `@property`

Phase 1 · Accounts and role gates — NEXT

Registration for families, login and logout, and a dashboard that routes each role to its own view. A reusable mixin that refuses a request from the wrong role.

CONCEPTS built-in auth views · `UserCreationForm` subclassing · `LoginRequiredMixin` · writing your own `UserPassesTestMixin` · messages framework · `{% csrf_token %}` and what it defends

DONE WHEN a family account signing in lands on the family dashboard and a hand-typed `/verifier/queue/` URL returns 403.

Phase 2 · Missing-person reports — THE FAMILY SIDE

The `MissingPersonReport` model, the four-step wizard, photo upload, a list of the user's own reports and a detail page with a status timeline.

CONCEPTS field types and `null` vs `blank` · `ModelForm` and the `Meta` inner class · class-based views (`CreateView` , `ListView` , `DetailView`) · `get_queryset()` filtered to `self.request.user` · `ImageField` , Pillow, `MEDIA_ROOT` · `get_absolute_url`

DONE WHEN a family can file a report with a photo and cannot open another family's report by changing the id in the URL.

Phase 3 · Unidentified records — THE RESPONDER SIDE

`Organisation` and `UnidentifiedRecord` . Responder intake form, and a list scoped to the responder's own organisation.

CONCEPTS `ForeignKey` , `related_name` and reverse lookups · `on_delete` choices and what each means · queryset filtering as an access-control mechanism · `select_related` and the N+1 query problem · custom model managers

DONE WHEN two responders in different organisations see disjoint lists, proven by a test.

Phase 4 · The matching engine

THE HEART

`MatchCandidate` as a through model. A pure-Python scoring module — no Django imports, so it is trivially unit-testable — and a management command that runs it and writes candidates above threshold.

CONCEPTS `ManyToManyField(through=...)` · `unique_together` / `UniqueConstraint` · `JSONField` for the score breakdown · custom management commands · `bulk_create` · separating domain logic from the framework

DONE WHEN `python manage.py run_matching` populates the candidate table and the scoring module has unit tests covering each rule.

Phase 5 · Verifier queue and comparison

THE DECISION

The ranked queue with its tabs, the side-by-side screen, computed agreements and disagreements, and the confirm / reject / claim transitions.

CONCEPTS complex querysets — `annotate`, `Q` objects, `order_by` · `transaction.atomic` · `select_for_update` against two verifiers claiming at once · state transitions guarded on the model, not in the view · template inclusion tags

DONE WHEN confirming a match closes both records and supersedes every sibling candidate, atomically.

Phase 6 · Notifications

Email and SMS to the family on a state change, and to verifiers when a high-score candidate appears. Queued, retried, and never sent twice.

CONCEPTS signals and when *not* to use them · email backends and the console backend in development · Celery or django-q with Redis · idempotency · templated messages

DONE WHEN a confirmation queues exactly one notification, even if the confirm button is double-clicked.

Phase 7 · Security hardening

YOUR TRACK

Private media behind a permission-checked view, the append-only audit log, rate limiting on the report form, and a full pass over Django's deployment checklist.

CONCEPTS serving files through a view with `FileResponse` and `X-Accel-Redirect` · middleware for audit capture · `django-axes` or custom throttling · `manage.py check --deploy` · security headers, HSTS, secure cookies · IDOR as a class of bug and how queryset scoping kills it

DONE WHEN `check --deploy` is clean and a signed-out request for a photo URL returns 403, not the image.

Phase 8 · PostgreSQL and real search

Migrate off SQLite. Trigram similarity and full-text search for the transliteration problem, and database indexes where the queue actually hurts.

CONCEPTS `django.contrib.postgres` · `TrigramSimilarity`, `SearchVector`, `SearchRank` · `GinIndex` · `Meta.indexes` · reading a query plan · why the ORM is not a substitute for understanding the database

DONE WHEN searching “Shreshtha” finds a record filed as “Shrestha”.

Phase 9 · API and public statistics

A read-only DRF API so a partner organisation could push records in, token-authenticated and scoped. A public dashboard of aggregate counts by district — figures only, never records.

CONCEPTS DRF serializers, viewsets, routers · token vs session authentication · per-view permission classes · pagination and throttling · `values()` with `annotate(Count(...))` for aggregates · charting from a JSON endpoint

Phase 10 · Seed data, tests, deployment

A fixture generator producing several hundred plausible fictional reports and records with known planted matches — that is how you prove the engine works. Test suite over the permission matrix. Docker, and a deploy.

CONCEPTS `factory_boy` / Faker · `TestCase` vs `TransactionTestCase` · testing permissions as a first-class concern · `coverage` · `DEBUG=False`, static file collection, WhiteNoise · environment-split settings

DONE WHEN a fresh clone reaches a working, populated site in three commands, and the planted matches are all found.

7 · Design principles to hold on to

- **The machine proposes, a person decides.** No automatic confirmation at any score. 99 is still a candidate.
- **Families never browse the dead.** No public gallery, no family-facing search across remains, no photograph without a verifier's deliberate act.
- **Show the reasoning, not just the score.** A verifier deciding on a number they cannot interrogate is a verifier who cannot be held responsible for the decision.
- **Blank beats guessed.** The forms should make “I don't know” easy, because a confident wrong height buries the right match.
- **Every read of a sensitive record is a logged event.** Not only every write.
- **Honest status, never false comfort.** “Searching” is true. “Not found” is a verdict the system has not earned.
- **Ownership is enforced in the queryset, not the template.** Hiding a link is not access control.

What to do next

Phase 0 is on disk. Run the setup commands, confirm the landing page and the admin, then start Phase 1 — registration, login, and the role gate. That gate is worth doing carefully: every screen in section 5 sits behind it, and getting it right once means never thinking about it again.

Sources

Kathmandu Post — *In search of the missing: families fan out across Nepal after Bhotekoshi flood*, 31 Aug 2026 · myRepublica — *Coordination gap hampers flood rescue, relief works* · EarthSky — *Nepal flash flood nightmare: what caused it?*, glacier collapse analysis · CBS News — flood toll and rescue conditions, Aug 2026 · UNDP Asia-Pacific — *Nepal Floods 2026*, RAPIDA assessment framework · IFRC — emergency appeal, isolated communities · Wikipedia — *2026 Nepal floods*.

> **REMINDER**
>
> Seeded fictional data only. This document describes a design; it is not an operational system, and it should not be presented as one.